

DOCUMENTO METODOLÓGICO

Modelo de Propensión a Aceptación de Opciones de Pago y Sistema Agéntico para
la Gestión de Cartera en Mora

Prueba Técnica — Bancolombia

Jose Nelson González

Septiembre de 2026

Tabla de Contenido

1. Introducción y Contexto de Negocio

Bancolombia gestiona un portafolio de crédito en el que un porcentaje de obligaciones entra en mora cada mes. Desde el primer día de mora se activan estrategias de gestión, entre ellas las opciones de pago (ampliación de plazo, reducción de cuota, renegociación de tasa, reestructuración) y los acuerdos de pago de gestión temprana (compromiso a máximo 5 días). Hoy la priorización de a quién gestionar se basa en exposición de la deuda y probabilidad de pago; el objetivo de esta prueba es incorporar, con un mes de anticipación, la probabilidad de que el cliente ACEPTE una opción de pago, y usarla dentro de un sistema de inteligencia artificial que decida y ejecute la siguiente mejor acción respetando las reglas de negocio.

Este documento consolida la metodología, las decisiones técnicas, los supuestos y los resultados de las dos partes de la prueba: (1) el modelo analítico de propensión y (2) el sistema agéntico de gestión proactiva y reactiva de clientes en mora.

2. Objetivo y Alcance

- Diseñar y desarrollar una solución analítica E2E que genere, por obligación, un indicador de propensión a aceptar una opción de pago en el mes siguiente, contemplando las etapas de MLOps.
- Proponer e implementar un prototipo funcional de un sistema de agentes de IA coordinados que gestione proactiva y reactivamente a los clientes en mora, ofreciendo únicamente lo que la política de crédito autoriza.
- Documentar metodología, supuestos, riesgos, pruebas y una propuesta de arquitectura de operación en producción para ambos componentes.

3. Documento Técnico — Cuerpo Principal

Nota: esta sección corresponde al documento técnico exigido por el enunciado (máximo 4.000 caracteres, excluyendo anexos, imágenes y tablas). El resto del documento (secciones 4 a 9) son los anexos de soporte con el detalle metodológico completo.

3.1 Parte 1 — Modelo de propensión

Proceso: se recibieron 4 tablas (trtest, master_customer_data, probabilidad_oblig_hist, maestra_cuotas_pagos_mes_hist) y oot.csv (enero-2024, solo IDs). El EDA mostró que la mayoría de columnas de trtest (gestiones, pagos, alternativa aplicada, mora fin de mes) no están disponibles en oot.csv y, además, muestran una correlación contemporánea muy alta con el target, consistente con que describen eventos del mismo mes que se quiere predecir (detalle y evidencia en Anexo A, sección 4.2). Se rediseñó el pipeline para usar solo información de t-1 hacia atrás (lags propios de la obligación, snapshot demográfico más reciente $\leq$ corte, scores históricos del banco), replicando el escenario real de pronosticar un mes antes.

Decisiones clave: se compararon 3 familias de modelos sobre el mismo split (lineal, bagging, boosting) — ver Anexo A, sección 4.5 — y se seleccionó LightGBM por mayor AUC de validación con margen claro, confirmado por F1; validación temporal (train ago-nov 2023, valid dic-2023, no aleatoria); umbral optimizado para F1. Resultado con las 4 tablas integradas: AUC=0.729, F1=0.671 en validación (el historial de pagos fue la incorporación de mayor impacto).

Supuestos: snapshot demográfico de dic-2023 usado como "as-of" para enero-2024 (el panel no llega a esa fecha); nulos demográficos (~40-50%) tratados como missing informativo, no imputados.

Riesgos/limitaciones: oot.csv no trae producto/banca/elegibilidad vigente, por lo que ~50% de sus obligaciones son "cold start" y dependen solo de demografía + scores del banco; el panel demográfico es disperso; se validó PSI dic-2023 vs. oot=0.013 (sin cambio poblacional relevante); SHAP de oot coincide 8/10 con validación (ver Anexo A, secciones 4.7 y 4.8).

Dato adicional recomendado: un snapshot de elegibilidad/producto vigente al momento del scoring (ausente hoy en oot.csv) mejoraría el modelo para obligaciones nuevas; costo bajo, pues el motor de preaprobación ya genera esa información mensualmente.

3.2 Parte 2 — Sistema agéntico

Arquitectura: patrón supervisor lineal y auditable — Contexto → Reglas de negocio (única fuente de verdad sobre qué ofrecer, determinística) → Siguiendo Mejor Acción (integra el score de la Parte 1) → Conversacional → Guardrails → Escalamiento, con trazabilidad JSON por sesión. En un dominio regulado, la previsibilidad de un flujo lineal pesa más que la flexibilidad de un grafo de agentes libre.

Sin acceso a LLM en este entorno: intención y redacción se implementaron con reglas léxicas/plantillas, con punto de extensión explícito para reemplazar por un LLM real sin tocar el motor de reglas — el LLM redactaría, nunca decidiría qué ofrecer.

Pruebas: 49 pruebas automatizadas (42 dirigidas + 7 masivas sobre 400 casos sintéticos, 100% pasan, cobertura de código ~100% en los 7 módulos de decisión/seguridad) + 14 escenarios dirigidos cubriendo los 7 casos pedidos más robustez y seguridad. Umbral: 0% de ofertas no autorizadas y 0% de restricciones ignoradas (tolerancia cero). Probar el sistema de punta a punta —no solo cada capa por separado— encontró y corrigió 3 problemas reales antes de la entrega (detalle en Anexo C, sección 6.3).

Riesgos: el NLU por reglas es frágil ante lenguaje real no anticipado; la priorización entre alternativas usa un orden fijo por severidad de mora (supuesto a validar), no un modelo aprendido — se investigó esta última opción con los datos reales de la prueba y no es viable con lo entregado (detalle en Anexo B, sección 5.6).

Conclusión general: ambos componentes son viables como prototipo demostrable dentro del alcance y tiempo de la prueba. El mayor riesgo de negocio no es el desempeño puntual del modelo sino mantener la disciplina de auditar qué variables están realmente disponibles al momento de decidir — un punto válido tanto para la Parte 1 como para su integración con la Parte 2.

3.3 Declaración de Uso de IA Generativa

Se usó Claude (Anthropic) como asistente de desarrollo de extremo a extremo: análisis exploratorio, identificación de variables no disponibles en oot.csv y con señal contemporánea alta frente al target, diseño del pipeline de features, entrenamiento del modelo, diseño de la arquitectura agéntica, e implementación de reglas/agentes/pruebas/documentación. El candidato dirigió el alcance, las decisiones de negocio (cooldowns, prioridades, qué construir dado el tiempo disponible) y revisó los resultados y supuestos antes de la entrega.

4. Anexo A — Metodología Detallada (Parte 1)

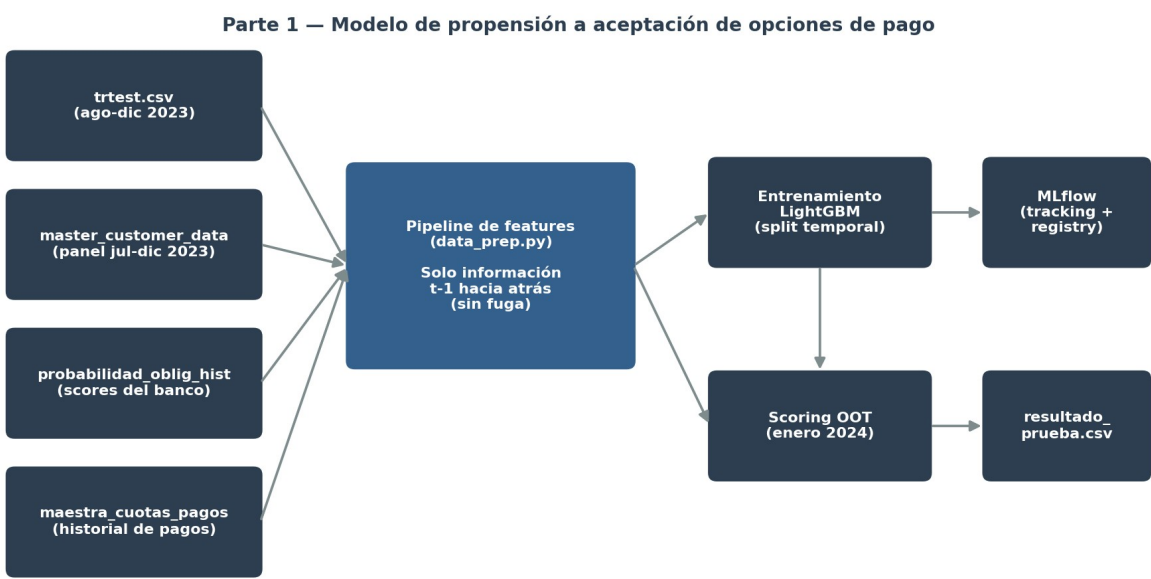
4.1 Datos disponibles

Tabla	Filas	Periodo	Llave
trtest.csv	568.251	Ago-Dic 2023	nit#num_oblig_orig#num_oblig_mes
master_customer_data.csv	430.000	Jul-Dic 2023 (panel)	nit, año-mes
probabilidad_oblig_hist.csv	4.804.836	Ene-Dic 2023 (panel)	num_oblig, año-mes
maestra_cuotas_pagos_mes_hist.csv	4.855.035	Ene-Dic 2023 (panel)	num_oblig, año-mes
oot.csv / sample_submission.csv	112.549	Enero 2024	nit#num_oblig_orig#num_oblig

4.2 Variables excluidas: disponibilidad en oot y consistencia temporal

Al comparar trtest.csv contra oot.csv, la mayoría de las columnas de trtest (cant_gestiones, pago_mes, porc_pago_mes, marca_alternativa, dias_mora_fin, saldo_capital, entre otras) no están disponibles en oot. Adicionalmente, al revisar su correlación con la variable respuesta, estas mismas columnas muestran una correlación contemporánea inusualmente alta, consistente con que describen eventos ocurridos durante el mismo mes de la variable que se quiere predecir. Por ambas razones — no disponibilidad en oot y esa señal contemporánea difícil de justificar como predictiva — se decidió no usarlas en su versión del mismo mes: usarlas así habría producido un modelo con métricas artificialmente altas en validación, pero poco útil en producción, porque esa información no está disponible cuando hay que emitir el pronóstico un mes antes.

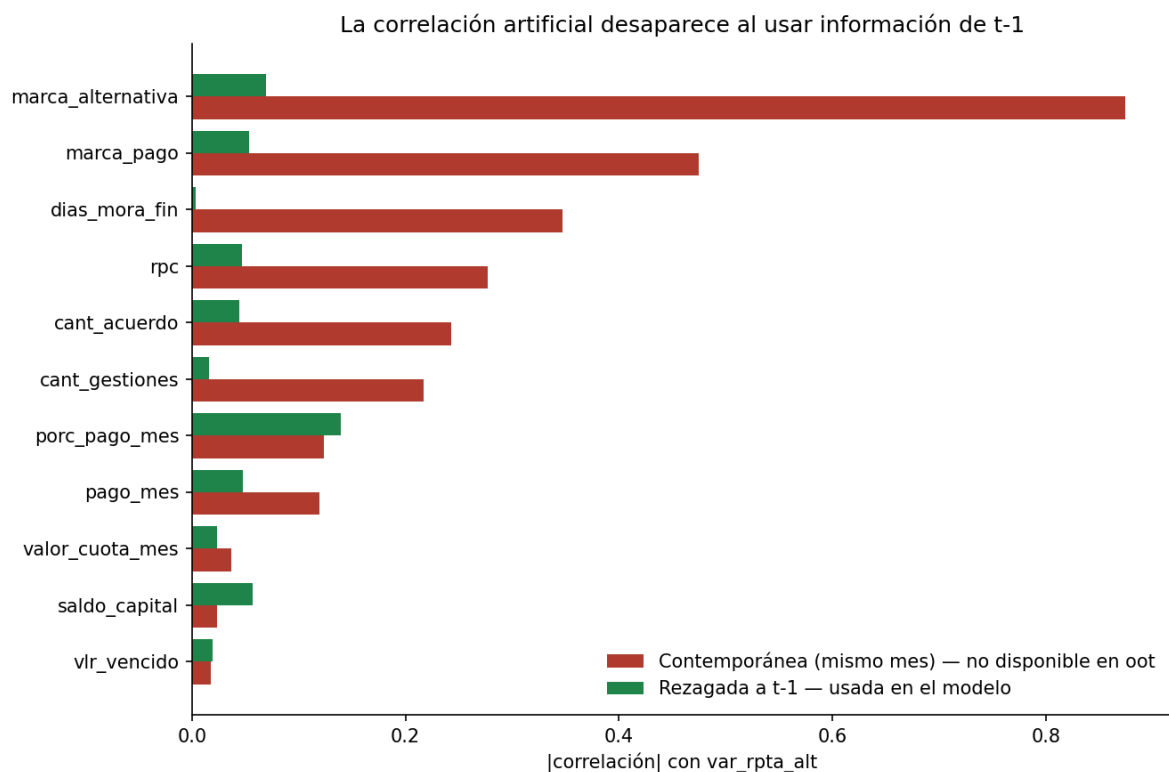
Decisión metodológica: estas columnas se usan únicamente como fuente de variables rezagadas (estado de la obligación en el mes t-1), nunca en su versión del mes t. Se conservan como contemporáneas solo las que describen la oferta/elegibilidad vigente (banca, segmento, producto, cantidad de alternativas preaprobadas) — aunque estas tampoco existen en oot.csv, por lo que el modelo final usa el subconjunto de 76 variables presentes en ambos mundos.



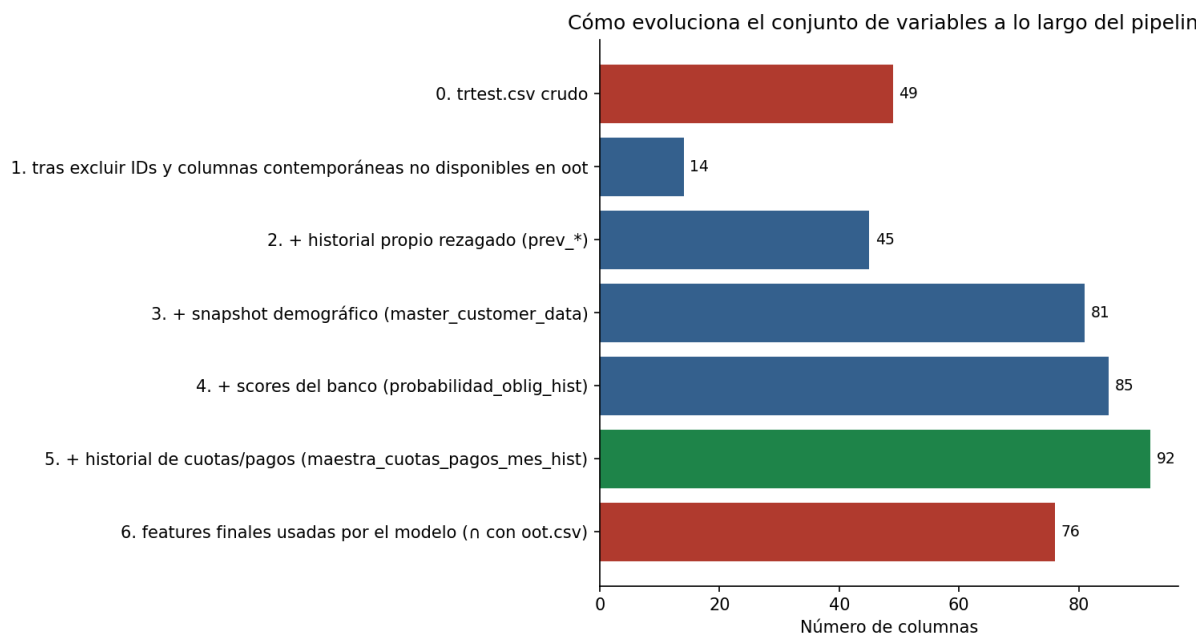
4.2.1 Evidencia de respaldo: correlación contemporánea vs. rezagada

notebooks/01_eda.py mide, para cada columna, su correlación con el target en versión contemporánea (mismo mes) contra su versión rezagada a t-1. La caída es notoria y es coherente con que la correlación contemporánea refleja información no disponible al momento real de la decisión, más que señal predictiva propia:

Variable	correlación contemporánea	correlación rezagada (t-1)
marca_alternativa	0.87	0.07
marca_pago	0.48	0.05
dias_mora_fin	0.35	0.00



El conjunto de variables también se depura progresivamente a medida que avanza el pipeline (49 columnas crudas → 14 tras excluir IDs y las columnas recién descritas → 45 con historial propio rezagado → 81 con demografía → 85 con scores del banco → 92 con historial de cuotas/pagos → 76 variables finales, intersección con oot.csv):



4.3 Variables y tratamiento de calidad de datos

- fecha_corte de maestra_cuotas_pagos_mes_hist viene en formato YYYYMMDD (no YYYYMM); se normalizó.
- porc_pago traía valores infinitos por división entre cuota=0; se limpiaron y se capó a 1000%.
- ~20% de los clientes de trtest nunca aparecen en master_customer_data; el resto tiene cobertura dispersa (promedio 1,78 snapshots en 6 meses) → nulos demográficos del 37-50%, tratados como missing informativo más una bandera tiene_snapshot_demografico.
- edad_cli contiene valores inválidos (0 y 123 años) reportados como hallazgo de calidad de datos, no corregidos por no afectar la variable de forma determinante dado el manejo nativo de nulos/outliers de LightGBM.

4.4 Validación y resultados

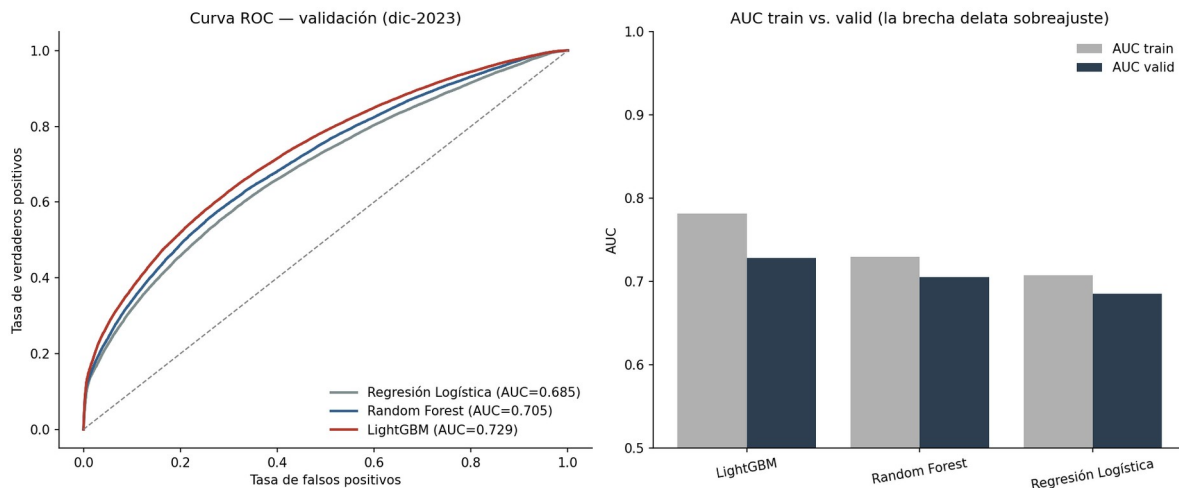
Split temporal (no aleatorio): entrenamiento con agosto-noviembre 2023, validación con diciembre 2023, replicando el escenario real de predecir el mes siguiente. Umbral de decisión optimizado para F1 (0.325), en vez de 0.5 por defecto.

Métrica	Valor
AUC (validación dic-2023)	0.729
F1 (validación dic-2023)	0.671
Precisión	0.551
Recall	0.859
Observaciones de entrenamiento / validación	467.785 / 100.466
Variables utilizadas	76

4.5 Selección de modelo: por qué LightGBM y no otra alternativa

Para no elegir LightGBM "por defecto", notebooks/02_model_comparison.py entrena, sobre el mismo split temporal y el mismo conjunto de 76 variables, tres familias de algoritmos — lineal, bagging y boosting:

Modelo	AUC train	AUC valid	Brecha train-valid	F1 valid
Regresión Logística	0.708	0.685	0.022	0.649
Random Forest	0.730	0.705	0.025	0.660
LightGBM (seleccionado)	0.782	0.729	0.053	0.671



Criterio de selección: se prioriza el mayor AUC de validación; la brecha train-valid solo se usa como desempate si dos modelos quedan a menos de 0.005 de AUC entre sí (no es el caso: LightGBM saca 2.4 puntos de AUC sobre Random Forest, una diferencia clara, no ruido). LightGBM gana con margen en AUC y en F1 —la métrica real de evaluación de la prueba— pese a tener la brecha train-valid más alta de los tres, un nivel moderado y ya controlado activamente en el pipeline (feature_fraction/bagging_fraction=0.8, min_data_in_leaf=100, early stopping).

Nota de honestidad metodológica: la comparación favorece estructuralmente a LightGBM en un aspecto real, no accidental — maneja categóricas/nulos de forma nativa, mientras que Regresión Logística y Random Forest necesitaron one-hot + imputación (con ciu, 446 categorías, agrupado a las 15 más frecuentes por necesidad de esos dos modelos). Esa capacidad nativa es justamente relevante para estos datos, no un artefacto que deba corregirse.

4.5.1 Configuración de hiperparámetros

La configuración final de LightGBM surge de una búsqueda controlada y dirigida —no una grilla exhaustiva— apoyada en buenas prácticas conocidas para boosting sobre datos tabulares con riesgo de sobreajuste: regularización vía muestreo de filas y columnas, tamaño mínimo de hoja, y early stopping sobre el conjunto de validación:

Hiperparámetro	Valor final	Propósito
learning_rate	0.05	Paso de aprendizaje conservador
num_leaves	63	Complejidad del árbol
min_data_in_leaf	100	Evita hojas sobreajustadas a pocos casos
feature_fraction	0.8	Muestreo de columnas por árbol (regularización)

Hiperparámetro	Valor final	Propósito
bagging_fraction	0.8	Muestreo de filas por iteración (regularización)
early_stopping_rounds	50	Detiene el entrenamiento si valid no mejora
num_boost_round (máx.)	2000	Techo; mejor iteración real alcanzada: 370

Random Forest y Regresión Logística (los otros dos modelos comparados en la sección 4.5) usan igualmente una configuración fija con el mismo criterio de regularización (p. ej. Random Forest: `n_estimators=200`, `max_depth=12`, `min_samples_leaf=50`), sin una búsqueda adicional sobre ellos, dado que no fueron los modelos seleccionados.

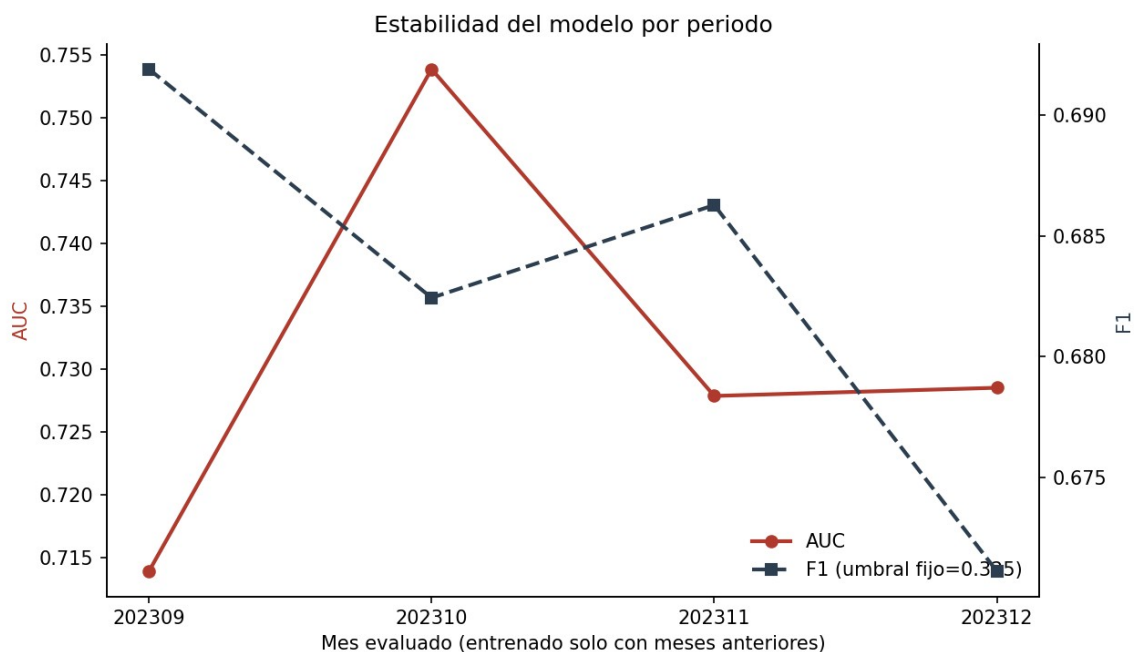
Oportunidad de mejora: esta búsqueda dirigida se puede ampliar a una grilla más exigente (grid search, random search u Optuna) sobre un rango más amplio de valores si se quisiera explorar el espacio de hiperparámetros de forma más sistemática — a costa de mayor capacidad de cómputo y tiempo de entrenamiento, no justificado dentro del alcance de esta prueba.

4.6 Selección final: estabilidad temporal e interpretabilidad

Con LightGBM ya seleccionado, se corren dos análisis adicionales sobre ese mismo modelo (dentro del mismo script, para no duplicar lógica de entrenamiento):

4.6.1 Estabilidad temporal (backtesting de ventana expansiva)

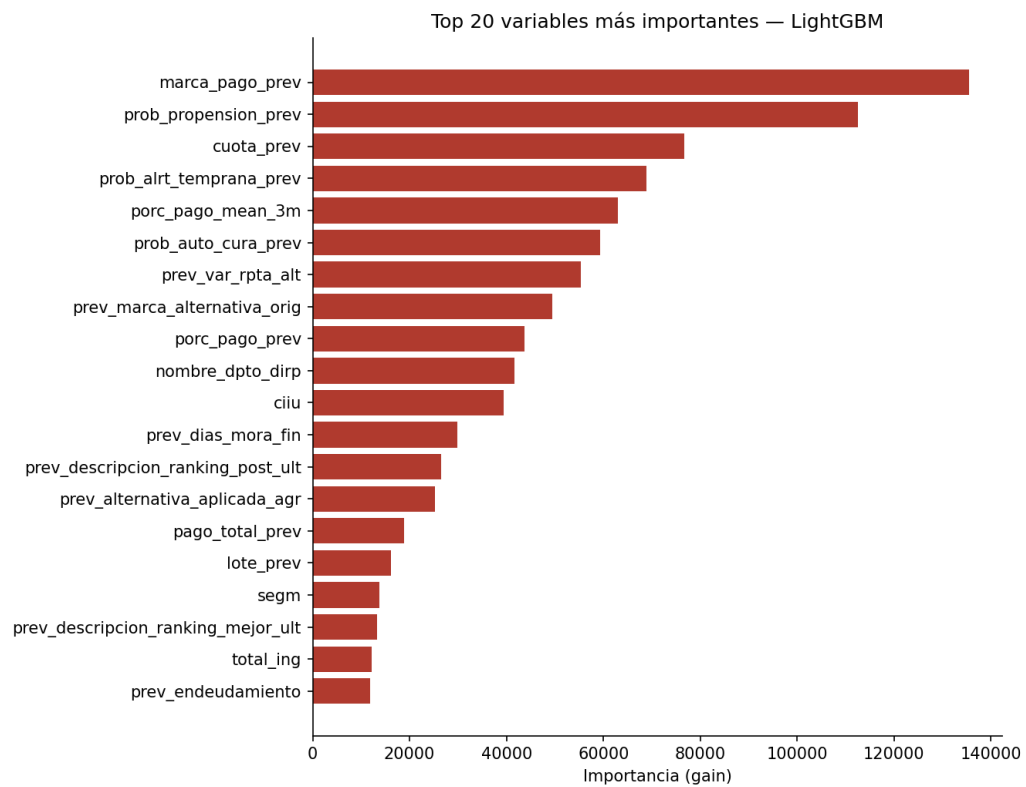
En vez de confiar en un único mes de validación, para cada mes desde septiembre se entrena solo con los meses anteriores y se mide AUC/F1 en un mes que el modelo nunca vio en ese entrenamiento. Resultado: AUC entre 0.714 y 0.754 en los 4 meses evaluados (media 0.731, desviación estándar 0.017) — el modelo es estable en el tiempo, sin señales de degradación abrupta mes a mes.

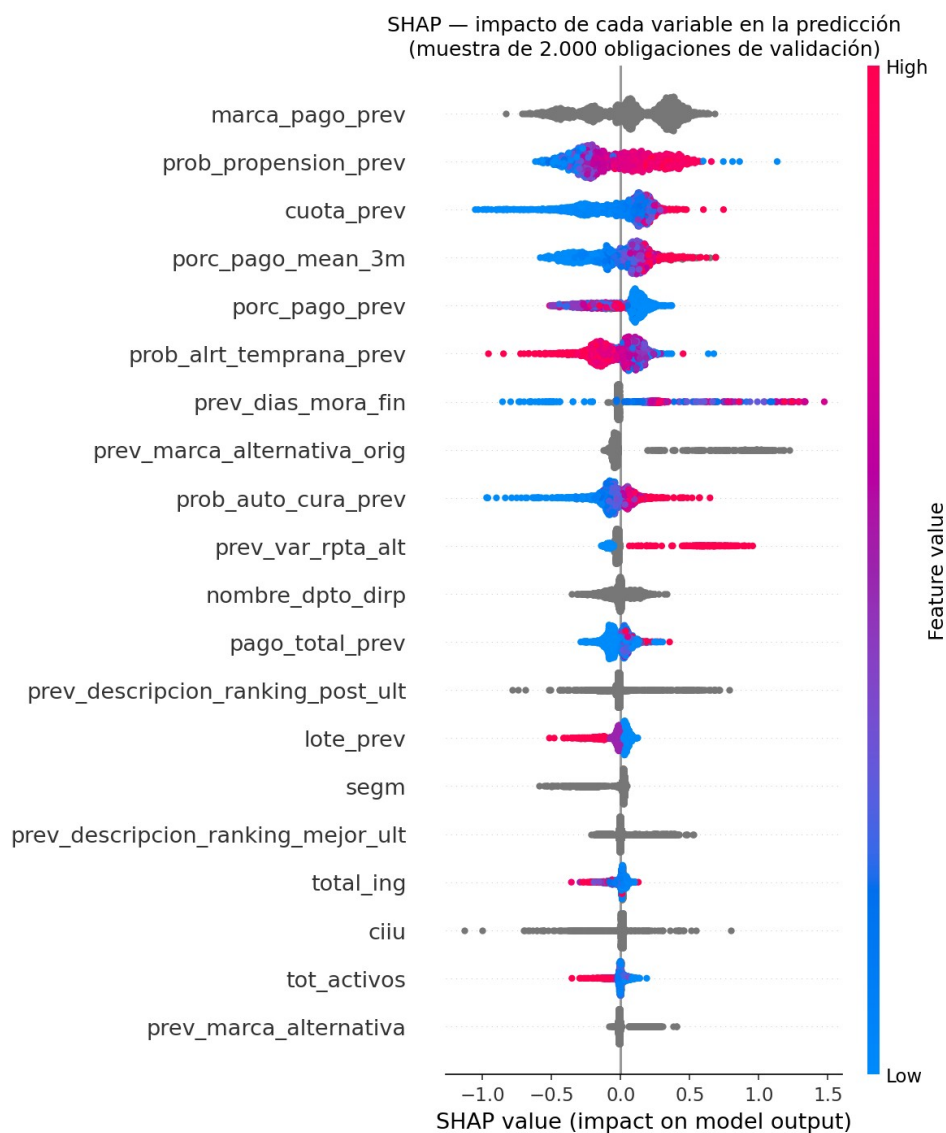


4.6.2 Interpretabilidad: importancia de variables y SHAP

El ranking de importancia por ganancia confirma lo reportado en la sección 4.7: `marca_pago_prev`, los 3 scores del banco, `cuota_prev` y `porc_pago_mean_3m` dominan. El gráfico SHAP añade la DIRECCIÓN del efecto por observación: por ejemplo, mayor `prev_dias_mora_fin` (más mora el mes

anterior) empuja la probabilidad de aceptación hacia arriba, consistente con que un cliente con mora reciente esté más dispuesto a aceptar una opción de pago.





4.7 Variables más importantes (top 12, por ganancia)

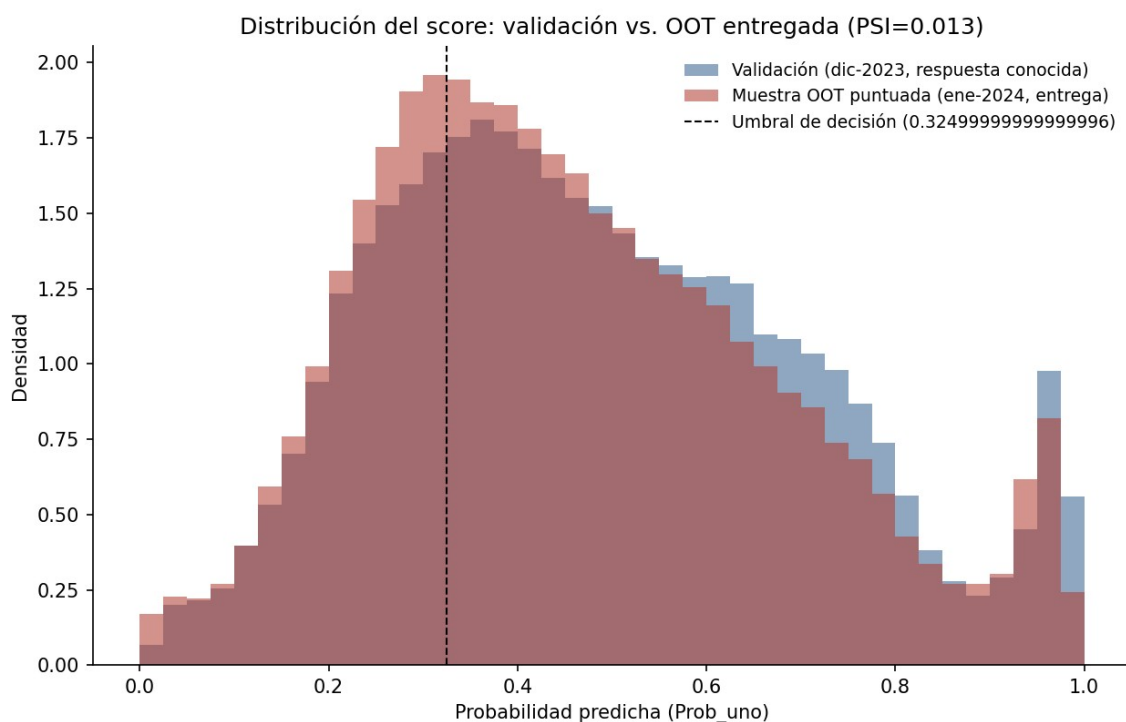
Variable	Descripción breve
marca_pago_prev	Tipo de pago realizado el mes anterior (más/menos/igual/no pagó)
prob_propension_prev	Score actual del banco: probabilidad de pago el mes siguiente
cuota_prev	Valor de la cuota del mes anterior
prob_alrt_temprana_prev	Score actual del banco: probabilidad de entrar en mora
porc_pago_mean_3m	Promedio de cumplimiento de pago en los últimos 3 meses
prob_auto_cura_prev	Score actual del banco: probabilidad de autocorrección
prev_var_rpta_alt	Si aceptó una opción de pago el mes anterior
prev_marca_alternativa_orig	Si aceptó originalmente una alternativa el mes anterior
porc_pago_prev	Porcentaje de pago de la cuota del mes anterior
nombre_dpto_dirp	Departamento de vinculación del cliente
ciiu	Actividad económica del cliente

Variable	Descripción breve
prev_dias_mora_fin	Días de mora al cierre del mes anterior

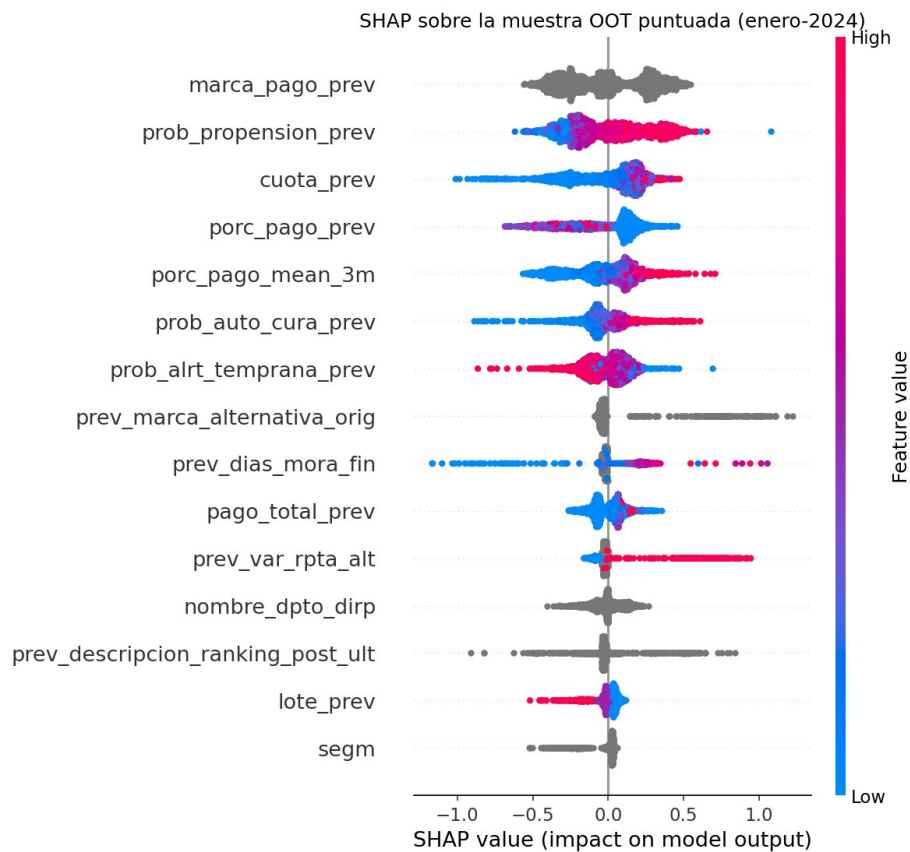
4.8 Validación de la muestra OOT entregada (resultado_prueba.csv)

resultado_prueba.csv NO es una simulación: son las 112.549 obligaciones reales (enmascaradas) de oot.csv (enero-2024) puntuadas por el modelo final — distinto del set de validación interno (dic-2023, con respuesta conocida) usado para medir AUC/F1. notebooks/03_analisis_oot_puntuada.py valida que esta entrega no tenga un comportamiento raro, en tres frentes:

- Reproducibilidad: recalcula la predicción desde el modelo guardado (results/model_lgbm.txt) y confirma que coincide con resultado_prueba.csv con una diferencia máxima de $1.11e-16$ — el mismo número, salvo redondeo de punto flotante.
- PSI (Population Stability Index) entre el score de diciembre-2023 (validación) y el de enero-2024 (la OOT entregada): 0.013, muy por debajo del umbral de 0.1 — sin cambio poblacional relevante de un mes a otro. Las dos distribuciones se superponen casi por completo:



- SHAP sobre la OOT: el ranking de variables más influyentes en enero-2024 coincide en 8 de las 10 principales con el ranking visto en validación, y con la misma dirección de efecto — la explicación del modelo es coherente entre el mundo donde se validó y el mundo donde se aplica de verdad.



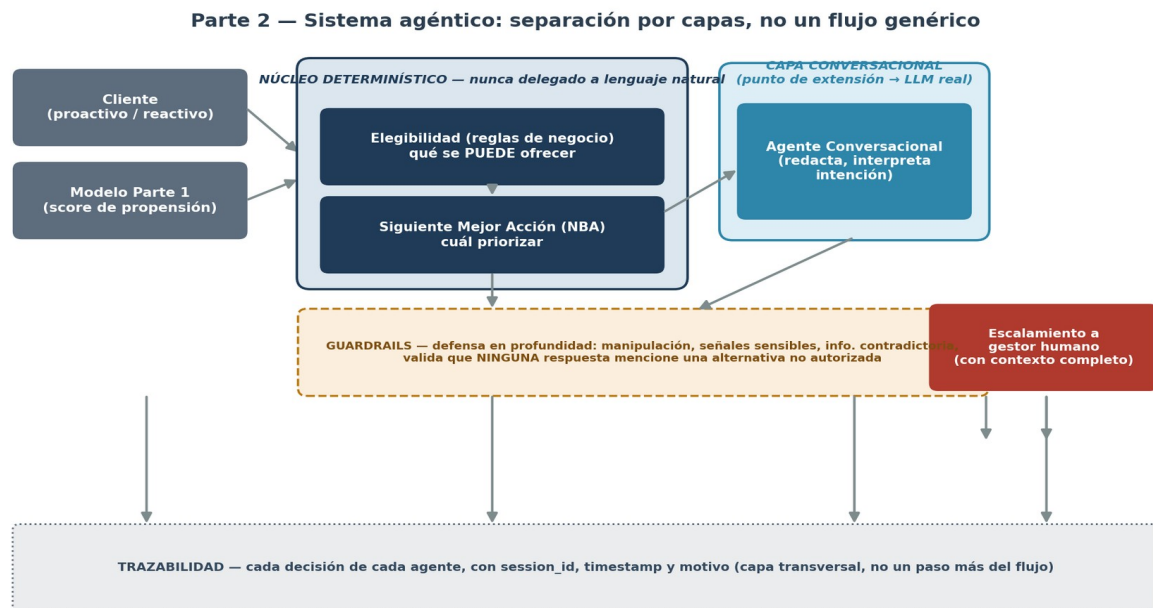
4.9 Cumplimiento de criterios de MLOps

Etapa MLOps	Implementación propuesta
Preparación de datos	Feature store con lógica punto-en-el-tiempo (paridad train/serve); validación de calidad con Great Expectations/pandera.
Entrenamiento	LightGBM + validación temporal; tracking de experimentos con MLflow; registro con etapas Staging/Production.
Inferencia	Batch mensual (alimenta la priorización por lotes) + endpoint on-demand (consultado por el sistema agéntico), ambos reutilizando el mismo pipeline de features.
Productización	Dockerfile + docker/requirements.txt IMPLEMENTADOS (empaquetan src/data_prep.py + src/train.py, datos/modelo montados como volumen, no horneados en la imagen); contrato de datos explícito y pruebas de contrato: propuestos.
Despliegue continuo	CI IMPLEMENTADO (.github/workflows/ci.yml): corre las 49 pruebas (42 dirigidas + 7 masivas) y valida el build de la imagen Docker en cada push/PR a main; CD con evaluación shadow y despliegue canario antes de promover: propuesto.
Monitoreo	Deriva de datos (PSI/KS), deriva de desempeño (F1/AUC real vs. esperado), calidad de servicio (latencia, cobertura de features), dashboards y alertas.

5. Anexo B — Arquitectura del Sistema Agéntico (Parte 2)

5.1 Filosofía de diseño

Cobranza es un dominio regulado: cada oferta debe ser exactamente la que la política de crédito autoriza, debe quedar trazada, y debe poder explicarse. Por eso se separa deliberadamente QUÉ se puede ofrecer (determinístico, motor de reglas, nunca delegado a un modelo de lenguaje) de CÓMO se comunica (lenguaje natural). Se eligió un patrón supervisor lineal y auditable en lugar de un grafo de agentes de diálogo libre: para este dominio, la previsibilidad pesa más que la flexibilidad.



5.2 Agentes y responsabilidades

Agente	Responsabilidad
Contexto	Ensambla el estado de la obligación: datos del cliente, mora, alternativas preaprobadas, historial y scores (Parte 1 + scores actuales del banco).
Elegibilidad (reglas de negocio)	Única fuente de verdad de qué se puede ofrecer: máx. 3 opciones/mes, cooldown 3-4 meses, bloqueo si ya hay opción vigente, restricciones duras.
Siguiente Mejor Acción (NBA)	Decide la acción concreta combinando elegibilidad + score de propensión + señales del banco (auto-cura, alerta temprana).
Conversacional	Redacta el mensaje y conduce el diálogo dentro de lo autorizado; interpreta intención del cliente.
Guardrails	Defensa en profundidad: detecta manipulación, señales sensibles, información contradictoria; valida la respuesta final.
Escalamiento	Punto único de handoff a gestor humano con el contexto completo.
Trazabilidad	Registra cada decisión de cada agente en un log estructurado por sesión.

5.3 Reglas de negocio clave

- Máximo 3 opciones de pago preaprobadas por obligación/mes.

- Cooldown de 3 meses (ampliación de plazo, reducción de cuota) o 4 meses (renegociación de tasa, reestructuración) tras aplicar una alternativa, antes de poder ofrecer otra (supuesto parametrizable).
- Si el cliente ya aceptó una opción de pago vigente, no se ofrece ninguna alternativa nueva ni acuerdo de pago.
- Acuerdo de pago (compromiso a máximo 5 días) solo en gestión temprana (mora $\leq$ 90 días) y sin restricciones activas.
- Cualquier restricción dura (jurídico, fraude, etc.) bloquea toda oferta y escala siempre a un gestor humano.
- Incumplimiento reciente ($\leq$ 90 días) de un acuerdo/opción de pago, detectado en el historial de gestión de la propia obligación: bloquea toda oferta nueva y escala, sin depender de que el cliente lo admita (ver sección 5.6.1).

5.4 Integración con el modelo analítico (Parte 1)

El score de propensión entra al Agente de Contexto como un campo más, obtenido del endpoint de inferencia. El NBA lo usa para: decidir si vale la pena contactar proactivamente, priorizar entre varias alternativas elegibles, y —junto con la probabilidad de auto-cura— diferir gestión intensa en mora muy temprana con alta probabilidad de autocorrección. El sistema es robusto a que este score no esté disponible: las reglas de negocio siguen operando de forma determinística (validado en pruebas de robustez, sección 6.1).

5.5 Seguridad, trazabilidad y escalamiento

- Todos los identificadores llegan enmascarados; el prototipo usa únicamente perfiles ficticios.
- Barrera de salida: valida que la respuesta generada nunca mencione una alternativa no autorizada, sin importar qué la generó.
- Cada sesión tiene un `session_id`; cada decisión de cada agente queda registrada con timestamp y motivo (explicabilidad no post-hoc).
- Restricciones duras, señales sensibles, manipulación, información contradictoria e incumplimiento reciente de un acuerdo previo escalan siempre a un gestor humano. El incumplimiento se detecta en dos capas: proactivamente sobre el historial estructurado, y como red de seguridad si el cliente lo admite en la conversación y el historial aún no lo refleja (rezago de datos).

5.6 ¿Por qué la priorización del NBA es una regla y no un modelo?

Una siguiente-mejor-acción "aprendida" —un modelo que, dado el perfil del cliente, prediga cuál de las alternativas elegibles tiene mayor probabilidad de ser aceptada, en vez de un orden fijo por severidad de mora— es el enfoque más sofisticado y el que se investigó explícitamente para esta prueba, no una idea descartada sin mirar.

5.6.1 Lo que sí hay en los datos

`trtest.csv` sí trae, por obligación-mes, hasta 3 alternativas candidatas (`desc_alternativa1/2/3`) y cuántas había disponibles (`cant_alter_posibles`) — exactamente el insumo que un modelo de este tipo necesitaría.

5.6.2 Por qué no se implementó aquí (verificado, no supuesto)

Se verificó si esas columnas están en oot.csv, la base que la prueba pide calificar: ninguna lo está (ni desc_alternativa1/2/3, ni cant_alter_posibles, ni las demás marcas de alternativa aplicada). Es la misma limitación ya documentada para producto/banca en la sección 4.9 — un modelo así se podría entrenar con trtest, pero no habría cómo aplicarlo ni validarlo sobre la muestra que esta prueba puntual exige entregar.

5.6.3 La dificultad adicional si se hiciera en producción

Incluso con esos datos disponibles, no se puede modelar ingenuamente "probabilidad de aceptar | se le ofreció la alternativa X" con el histórico tal cual: la alternativa ofrecida en el pasado no fue asignada al azar, sino por una regla o un asesor ya sesgada hacia el perfil del cliente. Habría que usar una técnica de tipo uplift/causal (efecto incremental de cada alternativa sobre la probabilidad de aceptación), no un clasificador directo, para no confundir "esta alternativa funciona mejor" con "esta alternativa se le dio a los clientes que de todas formas iban a aceptar".

Conclusión: queda como la oportunidad de mejora de mayor impacto potencial para producción (sección 6.5), condicionada a que el motor de preaprobación exponga las alternativas candidatas al momento del scoring — hoy ausentes de la base de calificación de esta prueba.

6. Anexo C — Pruebas y Validación del Sistema Agéntico

Por qué se prueba distinto que el modelo de la Parte 1: el modelo predice, así que se evalúa con métricas de error y se acepta una tasa de falla. El sistema agéntico codifica políticas que el banco ya decidió: no predice nada, así que el criterio correcto es cero incidentes de cumplimiento, no una tasa de error. Por eso se exige 100% en reglas de negocio y seguridad, y por eso conviene probar varias capas juntas, no solo cada una por separado (ver sección 6.3).

6.1 Pruebas automatizadas: dirigidas y masivas

49 pruebas, 100% pasan, de dos tipos complementarios y deliberadamente distintos en su lógica. Cobertura de código (pytest --cov=agentic): 100% en los 7 módulos de decisión/seguridad (reglas de negocio, NBA, guardrails, orquestador, modelos, trazabilidad, generador de casos sintéticos), 99% en el agente conversacional (la única línea sin cubrir es un respaldo defensivo inalcanzable con las 5 acciones actuales).

6.1.1 Pruebas dirigidas (42)

Casos puntuales escritos a mano, pensados para validar un comportamiento conocido y servir además como especificación legible del sistema. Organizadas en 6 capas:

Capa	# pruebas	Qué garantizan
Reglas de negocio	8	Máx. 3 opciones/mes, cooldown respetado y liberado a tiempo, bloqueo si ya hay opción vigente, restricción dura siempre escala, incumplimiento reciente bloquea y escala (y se libera pasada la ventana).
NBA	5	Priorización correcta según mora, diferimiento por auto-cura, robustez ante caída del servicio de scoring, rama de acuerdo de pago.
Guardrails (seguridad)	9	Detección de manipulación, señales sensibles, información contradictoria; cero falsos positivos; bloqueo de identificadores internos filtrados por error.
NLU (clasificación de intención aislada)	5	Cada patrón léxico clasifica el mensaje en la intención correcta.
Agente conversacional (respuesta end-to-end)	8	La RESPUESTA generada es la correcta, no solo la etiqueta; interacción entre guardrails y NLU cuando ambos podrían aplicar.
Integración end-to-end (orquestador)	7	Cliente no elegible nunca recibe oferta; restricción jurídica escala sin ofrecer; manipulación detiene el flujo; regresión de reasignación de alternativa; incumplimiento (proactivo y por admisión del cliente) escala en ambas rutas.

6.1.2 Pruebas masivas (7 pruebas sobre 400 casos sintéticos)

Complemento a las pruebas dirigidas: con casos escritos a mano siempre queda la duda de si las garantías se sostienen también en combinaciones no anticipadas. `agentic/generador_aleatorio.py` genera 400 obligaciones sintéticas con una semilla fija (reproducible), variando aleatoriamente mora, saldo, alternativas preaprobadas (incluyendo casos con más de 3, a propósito), historial de aplicaciones y gestiones (incluyendo incumplimientos dentro y fuera de ventana), restricciones duras y scores. Es deliberadamente sintética y no usa `trtest.csv`: la suite del sistema agéntico sigue sin

depender de los datos confidenciales de la Parte 1 y corre igual en cualquier equipo o en GitHub Actions. Sobre esa muestra se verifican 5 invariantes de negocio que deben cumplirse siempre, sin una sola excepción:

Invariante verificada sobre los 400 casos	Resultado
Restricción dura → siempre escala, cero alternativas ofrecidas	0 violaciones
Incumplimiento reciente (≤ 90 días) → siempre escala, cero ofertas	0 violaciones
Opción de pago ya vigente → nunca recibe una nueva oferta	0 violaciones
Ninguna alternativa ofrecida está en cooldown	0 violaciones
Nunca quedan más de 3 alternativas elegibles	0 violaciones

Más una prueba de forma que protege contra que a futuro se agregue una acción al NBA sin actualizar esta batería. El script `agentic/prueba_masiva.py` corre la misma simulación por fuera de `pytest` y guarda el detalle completo en `results/resumen_prueba_masiva.json`, útil como evidencia sin depender de correr la suite de pruebas.

6.2 Escenarios funcionales dirigidos (14)

Cubren explícitamente los 7 casos pedidos en el enunciado más 3 de robustez/seguridad. Perfiles y conversaciones 100% ficticios. 7 de 14 (50%) terminan en escalamiento — por diseño, para estresar cada gatillo, no como muestra representativa de producción.

#	Escenario	Resultado
1	Mora temprana + alta probabilidad de pago	Ofrece acuerdo de pago a 5 días
2	Elegible para varias opciones de pago	Prioriza y explica una alternativa
3	No elegible / cooldown activo	Sin ofertas (monitoreo)
4a	Rechaza la propuesta	Registra rechazo, no insiste
4b	Pide otra alternativa	Ofrece la siguiente elegible y registra la correcta al aceptar
4c	Incumplimiento ya en el historial de gestión	Escala de forma PROACTIVA, cero contacto
4d	Incumplimiento admitido solo por el cliente (historial aún no actualizado)	Escala igual, como red de seguridad conversacional
5a	Contacto reactivo: consulta de saldo	Responde directamente
5b	Contacto reactivo: dificultad financiera (pérdida de empleo)	Escala por señal sensible, antes del NLU
6	Información contradictoria	Escala para verificación humana
7a	Solicitud sensible (riesgo personal)	Escala de inmediato
7b	Intento de manipulación	Bloquea y escala
8	Restricción jurídica dura	Escala directo, cero contacto
9	Servicio de scoring caído	Sigue operando con reglas de negocio

6.3 Hallazgos de esta revisión: 3 bugs reales encontrados y corregidos

Ampliar las pruebas para cubrir el agente conversacional de punta a punta —no solo su clasificador de intención aislado— sacó a la luz tres problemas reales que ninguna prueba anterior cubría. Se

documentan de forma transparente porque son la evidencia más concreta de que la batería de pruebas agrega valor real:

- Bug de correctitud (el más serio): "no me sirve" (un rechazo) se clasificaba como ACEPTA, porque "me sirve" es subcadena y su patrón se evaluaba primero — el agente habría registrado que el cliente aceptó algo que en realidad rechazó. Corregido con un lookbehind negativo.
- Chequeo de seguridad sin efecto: la validación de la respuesta final recorría los identificadores internos en mayúsculas pero nunca usaba el resultado — no bloqueaba nada. Corregido: ahora si un mensaje filtra por error un código interno, se bloquea su envío.
- Regla de negocio incompleta: existía una función para detectar incumplimiento reciente que nunca se conectó — el sistema solo escalaba si el cliente lo admitía. Ahora se revisa proactivamente contra el historial de gestión (escenario 4c), dejando la detección conversacional como red de seguridad (escenario 4d).

Ninguno de los tres era visible probando cada capa por separado; los tres solo se manifiestan al combinar señales de punta a punta — la razón por la que la sección 6.1 ahora incluye una capa completa de pruebas del agente conversacional, no solo de clasificación de intención.

6.4 Métricas y umbrales de aceptación propuestos para producción

Métrica	Umbral propuesto
Tasa de ofertas no autorizadas	0% (bloqueo automático, no solo alerta)
Precisión del escalamiento	≥ 85% confirmado como necesario por un humano
Recall de señales sensibles	≥ 95% (el falso negativo es el error costoso)
Tiempo a escalamiento	< 5 segundos
Cobertura de intención (no ambiguo)	≥ 80%

6.5 Oportunidades de mejora identificadas

- Reemplazar el NLU basado en reglas léxicas por un LLM con salida estructurada, manteniendo las reglas de negocio como red de seguridad, no como reemplazo.
- Aprender la priorización entre alternativas elegibles con un modelo de tipo uplift, en vez del orden fijo actual por severidad de mora — investigado en la sección 5.6: requiere que el motor de preaprobación exponga las alternativas candidatas al momento del scoring, hoy ausentes de oot.csv.
- Incorporar pruebas de carga/concurrencia, fuera del alcance de este prototipo.
- Mantener la práctica de correr la suite completa de integración (no solo pruebas unitarias por capa) cada vez que se agregue o cambie un patrón léxico — es la única forma en que los hallazgos de la sección 6.3 se detectan antes de producción.

7. Anexo D — Arquitectura de Producción y Operación (Propuesta)

Visión end-to-end, independiente de proveedor cloud: ingesta y features (feature store con lógica punto-en-el-tiempo) → entrenamiento/registro (MLflow, con aprobación de promoción por comparación shadow) → scoring batch mensual (alimenta la priorización por lotes existente) y on-demand (consultado por el sistema agéntico) → orquestador agéntico (reglas → NBA → conversacional → guardrails) → cola de escalamiento humano con contexto completo → observabilidad (deriva de datos/desempeño, tasa de ofertas no autorizadas, LLMops del componente conversacional cuando se incorpore un LLM real).

- Seguridad: identificadores enmascarados end-to-end, RBAC sobre la cola de escalamiento y las trazas, cifrado y retención conforme a Habeas Data.
- Plan de rollback: reversión automática a la versión anterior del modelo/reglas si el monitoreo detecta ofertas no autorizadas > 0% o caída abrupta de F1 (feature flag).
- Mantenimiento: dueño de producto por componente, con revisión trimestral de los supuestos de negocio parametrizados (cooldowns, umbrales, prioridades) junto con el área de política de cartera.

8. Anexo E — Declaración de Uso de IA Generativa (detalle)

Herramienta utilizada: Claude (Anthropic), a través de Claude Code/Cowork, durante toda la prueba. El candidato guio la metodología a seguir en cada etapa, validando y contrastando los resultados frente a los supuestos y parámetros de la prueba.

Actividad	Rol de la IA generativa	Rol del candidato
Análisis exploratorio de datos	Ejecución de código de exploración e identificación de variables no disponibles en oot / con señal contemporánea alta	Validación de esa decisión y su relevancia para el negocio
Diseño del pipeline de features	Propuesta e implementación del diseño "as-of" (solo información conocida antes del mes a predecir)	Revisión de la lógica y de los supuestos de cooldown/ventanas
Entrenamiento y evaluación del modelo	Implementación, ejecución y reporte de métricas	Definición del criterio de éxito (F1) y revisión de resultados
Arquitectura y código del sistema agéntico	Diseño de agentes, reglas, guardrails, pruebas e implementación	Definición de reglas de negocio, alcance y priorización dado el tiempo disponible
Documentación	Redacción de todos los documentos (este incluido)	Revisión, ajuste y validación del contenido antes de la entrega

El candidato además replicó el pipeline completo (Parte 1 y Parte 2) en su propio entorno local, paso a paso, como verificación independiente de principio a fin antes de la entrega (ver README.md, sección "Cómo reproducir"). Ninguna sección de este documento fue tomada de fuentes externas sin adaptación al contexto específico de esta prueba; todo el código fue ejecutado y verificado antes de incluirse en la entrega.

9. Anexo F — Supuestos, Asunciones y Limitaciones (Consolidado)

Esta sección reúne en un solo lugar, por tema, los supuestos y limitaciones que ya se mencionan a lo largo del documento — no introduce hallazgos nuevos, es un punto único de referencia para quien revise la entrega. Cada punto indica dónde encontrar el detalle y la evidencia completa.

9.1 Supuestos del modelo estadístico (Parte 1)

- Supuesto general de todo modelo predictivo entrenado con datos históricos: se asume que el comportamiento reciente de los clientes y de la cartera es representativo de lo que va a ocurrir en el mes inmediatamente siguiente (enero 2024). Un cambio estructural repentino (choque macroeconómico, cambio de política de cartera, etc.) no estaría capturado por el modelo hasta que existan datos de ese nuevo régimen — de ahí el valor del monitoreo de deriva propuesto en el Anexo D.
- Snapshot demográfico de diciembre-2023 usado como "as-of" para enero-2024, porque el panel demográfico no llega a esa fecha (sección 4.1); es un supuesto explícito, no un dato real de enero.
- Nulos demográficos (~40-50%) tratados como missing informativo (bandera `tiene_snapshot_demografico`), no imputados (sección 4.3).

9.2 Variables excluidas por disponibilidad y consistencia temporal

- Varias columnas de `trtest.csv` no están disponibles en `oot.csv` y, además, muestran una correlación contemporánea con el target inusualmente alta — consistente con que describen eventos del mismo mes que se quiere predecir. Por ambas razones se excluyeron del conjunto contemporáneo y solo se usan en su versión rezagada ($t-1$). Detalle, tabla de correlaciones y evidencia gráfica en la sección 4.2 y 4.2.1.

9.3 Cobertura y calidad de datos

- `oot.csv` no trae producto/banca/elegibilidad vigente, por lo que ~50% de sus obligaciones son "cold start" y dependen solo de demografía + scores del banco (sección 4.4).
- El panel `master_customer_data` es disperso (promedio 1,78 snapshots por cliente en 6 meses); ~20% de los clientes de `trtest` nunca aparecen en él (sección 4.3).
- `edad_cli` contiene valores inválidos (0 y 123 años); se reportan como hallazgo de calidad de datos, no se corrigen, dado el manejo nativo de nulos/outliers de LightGBM (sección 4.3).
- `porc_pago` traía valores infinitos por división entre cuota=0; se limpiaron y se limitaron a 1000% (sección 4.3).

9.4 Configuración de hiperparámetros

- La configuración final de LightGBM surge de una búsqueda controlada y dirigida, no de una grilla exhaustiva. Ampliarla a una grilla más exigente (grid search, random search u Optuna) sobre un rango más amplio de valores es una oportunidad de mejora, a costa de mayor capacidad de cómputo y tiempo de entrenamiento (detalle y tabla de valores en la sección 4.5.1).

9.5 Supuestos y limitaciones del sistema agéntico (Parte 2)

- Sin acceso a un LLM real en este entorno: la redacción y la interpretación de intención se implementaron con reglas léxicas/plantillas, con un punto de extensión explícito para reemplazarlas por un LLM real sin tocar el motor de reglas (secciones 5.1 y 5.2).
- El NLU por reglas es frágil ante lenguaje real no anticipado por los patrones definidos (sección 6.3 documenta un bug real de este tipo, ya corregido, encontrado durante las pruebas).
- La priorización entre alternativas (Siguiente Mejor Acción) usa un orden fijo por severidad de mora — un supuesto de negocio a validar con el área de política de cartera, no aprendido de datos históricos de aceptación.

- Se investigó si esa priorización podía basarse en un modelo en vez de una regla fija; no es viable con los datos entregados en esta prueba (columnas de alternativas ofrecidas ausentes en oot.csv), y aun si lo fueran, requeriría una metodología de inferencia causal/uplift por el sesgo de selección en cómo se asignaron las alternativas históricamente — detalle completo en la sección 5.6.
- La arquitectura de producción descrita en el Anexo D es una propuesta de diseño, no una implementación — el prototipo de esta prueba corre en el entorno local descrito en la sección 10.

10. Estructura del Repositorio y Reproducibilidad

El repositorio Git entregado contiene:

- src/ — pipeline de datos (data_prep.py) y entrenamiento/inferencia (train.py) de la Parte 1.
- agentic/ — modelos, reglas de negocio, NBA, guardrails, conversacional, orquestador, escenarios dirigidos y generador de casos sintéticos para la prueba masiva de la Parte 2.
- tests/ — 49 pruebas automatizadas (pytest): 42 dirigidas + 7 masivas, con cobertura de código.
- docs/ — este documento y sus fuentes en Markdown (documento_tecnico.md, eda_notas.md, ml_ops_parte1.md, arquitectura_agentica.md, pruebas_agentico.md, arquitectura_produccion.md, presentacion_ejecutiva.md).
- results/ — resultado_prueba.csv, métricas, importancia de variables y trazas de las sesiones agénticas.

Instrucciones de reproducción completas en README.md del repositorio.